SIEMENS

Digital Twin

Capstone Project

Line-Following Robot with PID

Submitted to:

Eng. Mohamed Leithy

Date of Submission:

March 20th

Report by: Ammar Ahmed Wahidi

Contents

Introduction 2
System Architecture 2
Three-Client Architecture 2
IVSI Signal Mapping (CAN IDs) 3
Robot Kinematics and Modeling 3
Differential Drive Kinematics 3
Sensor Modelling 4
Disturbance Model 4
PID Controller Design 4
Experiment 1 – PID Gain Sweep 5
Experiment 2 – Curved Path 7
Experiment 3 – Noise and Disturbance 8
Noise 8
Disturbance 9
Experiment 4 – PD vs PID 10
Results 10
Conclusion 11
Githup Link 11

Introduction

This report presents the design, implementation, and experimental evaluation of a line-following robot digital twin using the Siemens Innexis Virtual System Interconnect (IVSI) platform. A differential-drive robot is simulated in a 2D environment and controlled using a PID (Proportional-Integral-Derivative) controller to follow a predefined reference path with minimal lateral error.

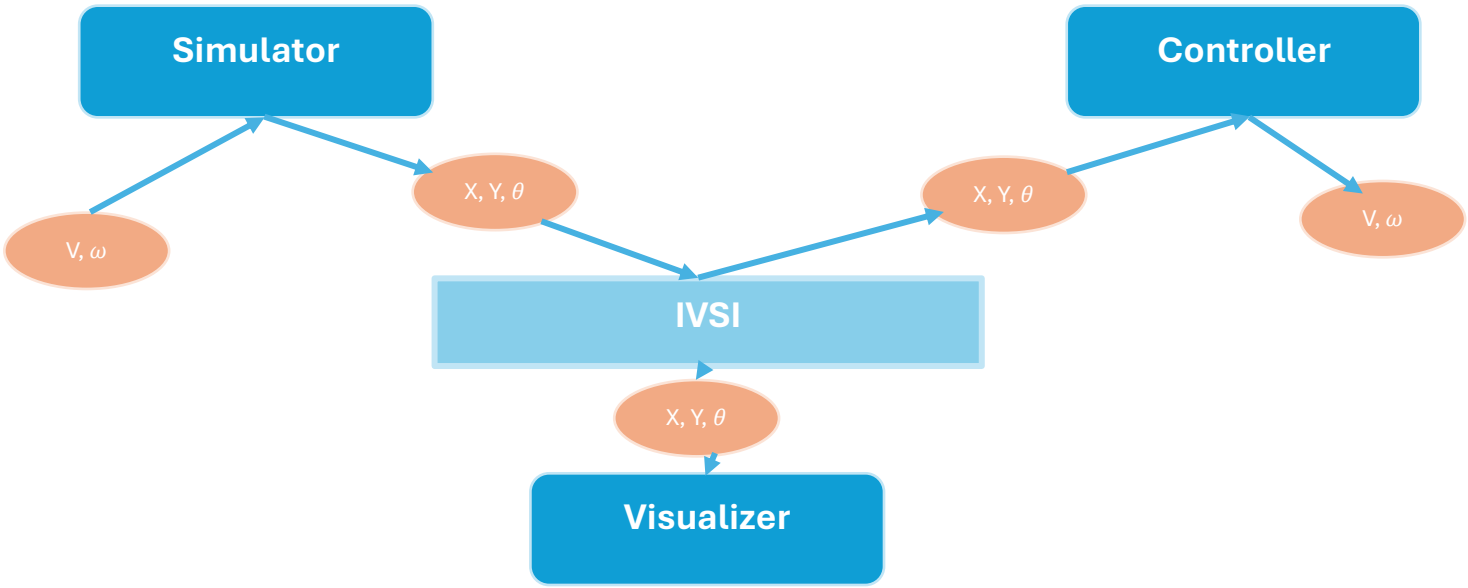
The system is implemented as three lightweight Python clients communicating over the IVSI CAN protocol backplane: a Simulator (plant), a Controller, and a Visualizer/Logger. Four mandatory experiments were conducted to evaluate controller performance across different gain sets, path types, noise levels, and controller configurations.

System Architecture

Three-Client Architecture

The system consists of three Python clients connected over the IVSI backplane using CAN protocol:

- Client 1 — Simulator (Plant + Environment): Simulates differential-drive robot kinematics, publishes robot pose (x, y, theta) to the IVSI backplane, and injects Gaussian noise and angular disturbances.
- Client 2 — Controller: Implements the PID control law, reads robot pose from IVSI, computes velocity commands (v, omega), and publishes them back to the backplane.
- Client 3 — Visualizer/Logger: Subscribes to robot pose signals, plots trajectory versus reference path, and computes KPIs (overshoot, settling time, steady-state error).



IVSI Signal Mapping (CAN IDs)

CAN ID	Signal	Direction	Description
10	x	Simulator **to** Controller, Visualizer	Robot X position (m)
11	y	Simulator **to** Controller, Visualizer	Robot Y position (m)
12	theta	Simulator **to** Controller, Visualizer	Robot heading angle (rad)
13	v	Controller **to** Simulator	Forward velocity command (m/s)
14	omega	Controller **to** Simulator	Angular velocity command (rad/s)

Robot Kinematics and Modeling

Differential Drive Kinematics

The total linear velocity of the robot center is the average of the two wheels:

$$v = \frac{v_r + v_l}{2}$$

The angular velocity is determined by the difference in wheel speeds:

$$\omega = \frac{v_r - v_l}{L}$$

To simulate the robot's trajectory, you update the pose over a small time step $\Delta \boldsymbol{t}$ using the following differential equations:

$$\dot{x} = v\cos(\theta)$$

$$\dot{y} = v\sin(\theta)$$

$$\dot{\theta} = \omega$$

In a discrete-time simulation, the next state is calculated as:

- $x_{t+1} = x_t + v\cos(\theta_t)\,\Delta t$
- $y_{t+1} = y_t + v\sin(\theta_t)\,\Delta t$
- $\theta_{t+1} = \theta_t + \omega\Delta t$

Sensor Modelling

Gaussian noise is added to simulate real sensor imperfections. The noise is applied to the x and y position states:

$$x_{noisy} = x + \sigma \times N(0,1)\Delta t\,, \qquad y_{noisy} = y + \sigma \times N(0,1)\Delta t$$

Where σ is the noise standard deviation (noise_level). Values of σ = 0.05 (low) and σ = 0.2 (high) were tested in E3.

Disturbance Model

Angular disturbances simulate sudden external pushes. With probability p = 0.001 per step, an impulse is applied:

$$\theta = \theta \pm d_{mag} \quad (p = 0.001 perstep)$$

Where d_mag is the disturbance magnitude. Values of 0.5 rad/s (low) and 2.0 rad/s (high) were tested.

PID Controller Design

The controller minimizes both lateral error (distance from path) and heading error (angle misalignment). The control output omega is computed as:

$$\omega(t) = \underbrace{K_p e_{lat}(t) + K_i \int_0^t e_{lat}(\tau)d\tau + K_d \frac{de_{lat}(t)}{dt}}_{\text{Lateral PID}} + \underbrace{K_{p_head} e_{heading}(t)}_{\text{Heading Correction}} + \underbrace{0.5\left(\frac{dy}{dx}\right)}_{\text{Feed-forward}}$$

Lateral PID (K_p, K_i, K_d): This focuses on the perpendicular distance (e_{lat}) from the path.

Heading Term $\left(K_{p_head} \cdot e_{heading}\right)$: This adds a proportional correction for the orientation error. Including this is highly effective for preventing the "snake-like" oscillations common in pure lateral controllers.

Feed-forward Term $0.5\left(\frac{dy}{dx}\right)$: This term uses the path gradient to "anticipate" curves. While useful for the E2-Curved Path experiment ,

Experiment 1 – PID Gain Sweep

Five gain sets were evaluated on a straight reference path (y = 0). For each gain set, three runs with random spawns were conducted. The best-performing gain set was selected for use in E2, E3, and E4.

Set	Kp	Ki	Kd	Overshoot (avg)	Settling Time (avg)	SS Error (avg)
1	0.2	0.0	0.01	0.8968 m	35.1 s	0.0144 m
2	0.5	0.02	0.05	0.3978 m	29.9 s	0.0225 m
3	1.0	0.05	0.08	0.3358 m	47.6 s	0.0000 m
4	1.5	0.05	0.10	0.2992 m	35.5 s	0.0000 m
5	2.0	0.1	0.15	0.3629 m	29.1 s	0.0000 m

Note: Simulation time is 200 s, must set to value that $0.1 \times \text{Overshoot}$ to calculate Setting Time

Set 5 (Kp=2.0, Ki=0.1, Kd=0.15) achieved the fastest settling time (29.1s) and zero steady-state error, making it the best performing gain set. Lower Kp values (Set 1) resulted in slower convergence (35.1s settling) with higher steady-state error, while higher Kp values (Sets 3-5) achieved zero steady-state error due to the integral term eliminating accumulated offset. The derivative term Kd helps dampen oscillations and prevent overshooting.

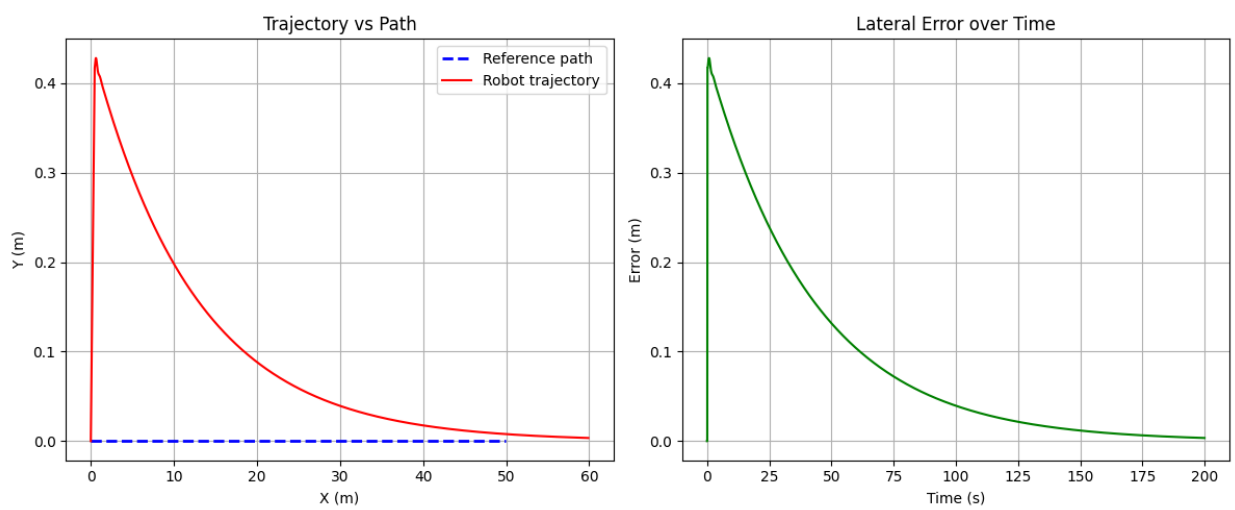


Figure 1 E1-Set1

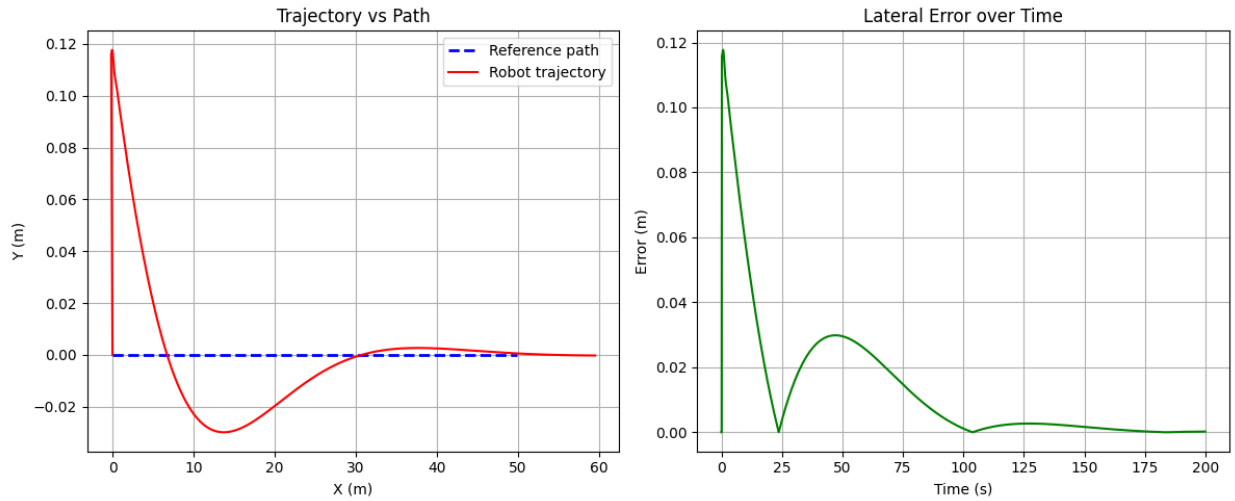


Figure 2 E1-Set2

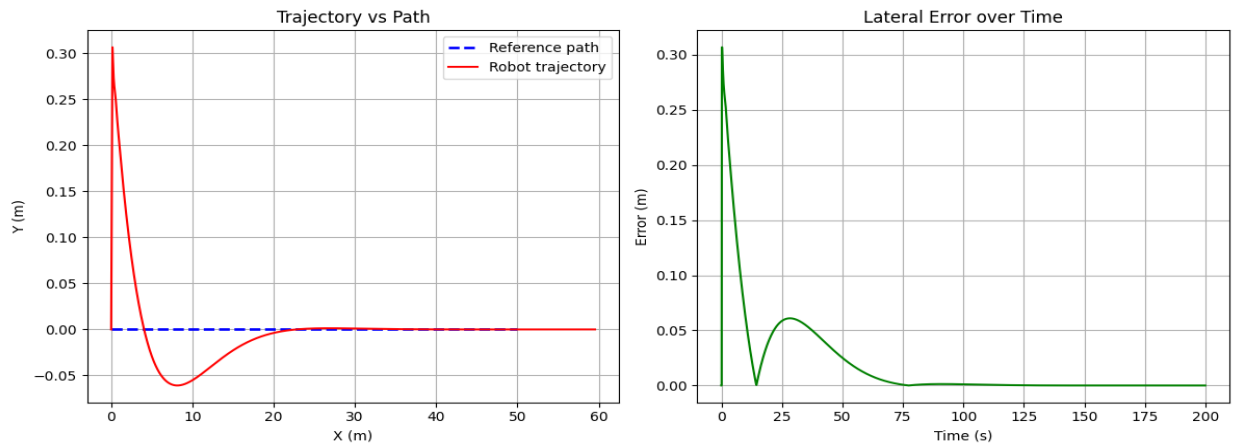


Figure 3 E1-Set3

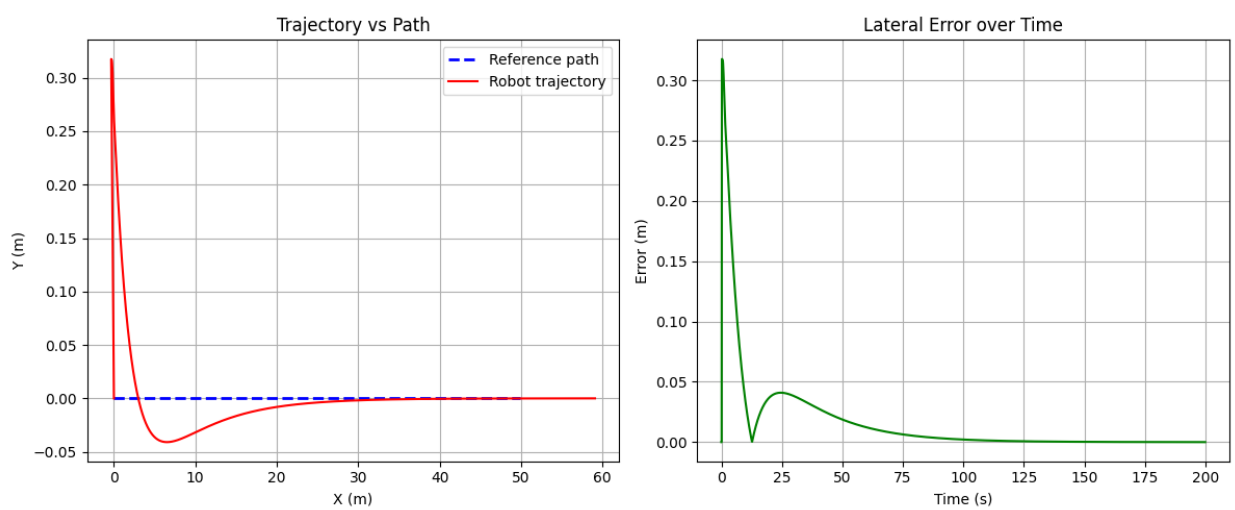


Figure 4 E1-Set4

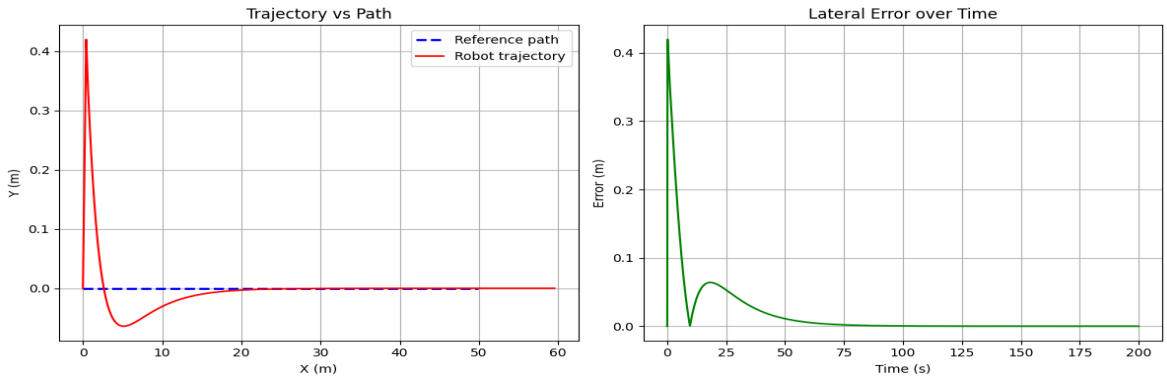


Figure 5 E1-Set5

Experiment 2 – Curved Path

The best gain set from E1 (Kp=2.0, Ki=0.1, Kd=0.15, Kp_head=2.5) was applied to a sinusoidal curved path defined as y_ref = 2.0 · sin(0.3 · x). Three random spawn runs were conducted and compared against the straight path results.

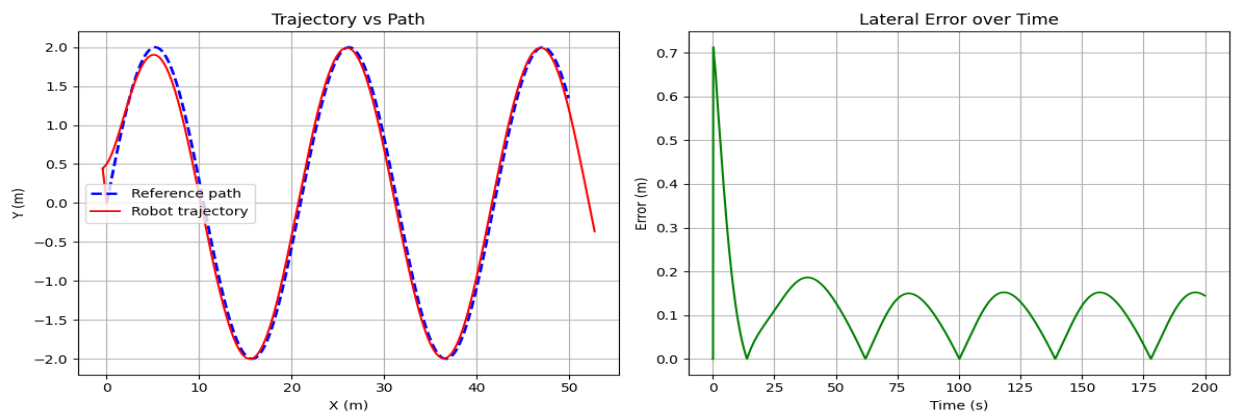


Figure 7 E2 Sin Curve Path

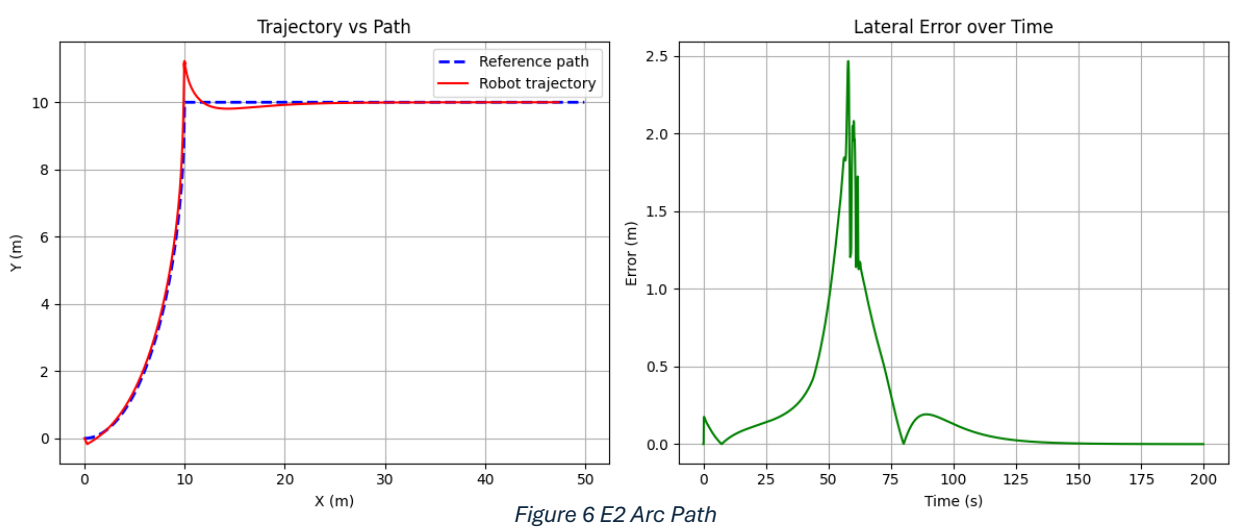


Figure 6 E2 Arc Path

Experiment 3 – Noise and Disturbance

The robustness of the best controller (Kp=2.0, Ki=0.1, Kd=0.15) was evaluated under four conditions: low noise, high noise, low disturbance, and high disturbance. Noise was applied as Gaussian position perturbations. Disturbances were applied as random angular impulses with probability 0.001 per step.

Noise

Condition	σ (noise)	Overshoot (avg)	SS Error (avg)	Success Rate
No noise (E1 baseline)	0.0	0.3629 m	0.000 m	3/3 = 100%
Low noise	0.05	0.3160 m	0.0170 m	3/3 = 100%
High noise	0.2	0.3223 m	0.0544 m	3/3 = 100%

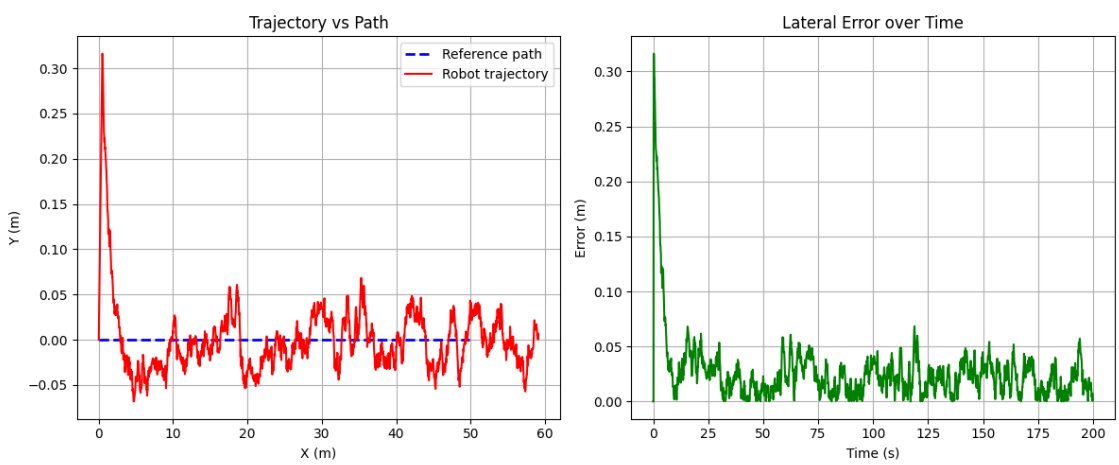


Figure 8 E3 Low Noise

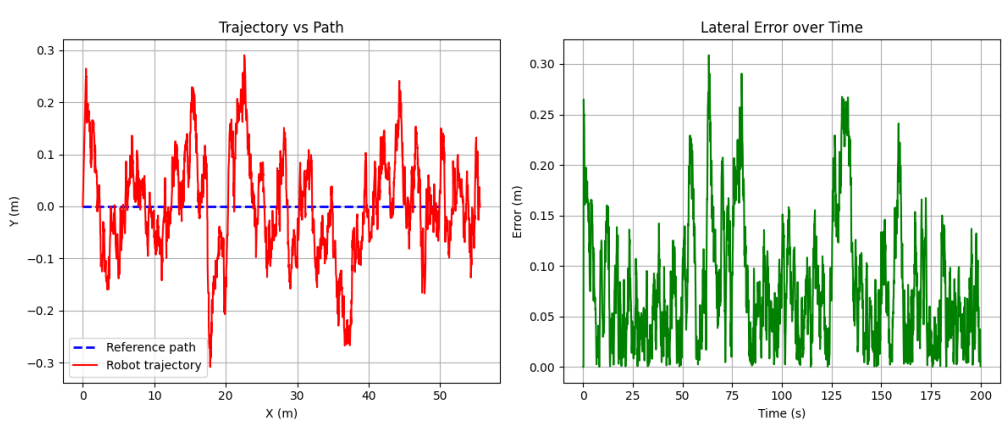


Figure 9 E3 High Noise

Disturbance

Low disturbance (d_mag = 0.5 rad/s): The robot experienced occasional sudden heading changes but recovered quickly and returned to the path. The trajectory plot shows sharp spikes in the error signal followed by rapid recovery.

High disturbance (d_mag = 2.0 rad/s): Larger angular impulses caused more significant deviations. The PID controller's heading correction term (Kp_head · e_heading) enabled recovery in all cases, though recovery time was longer.

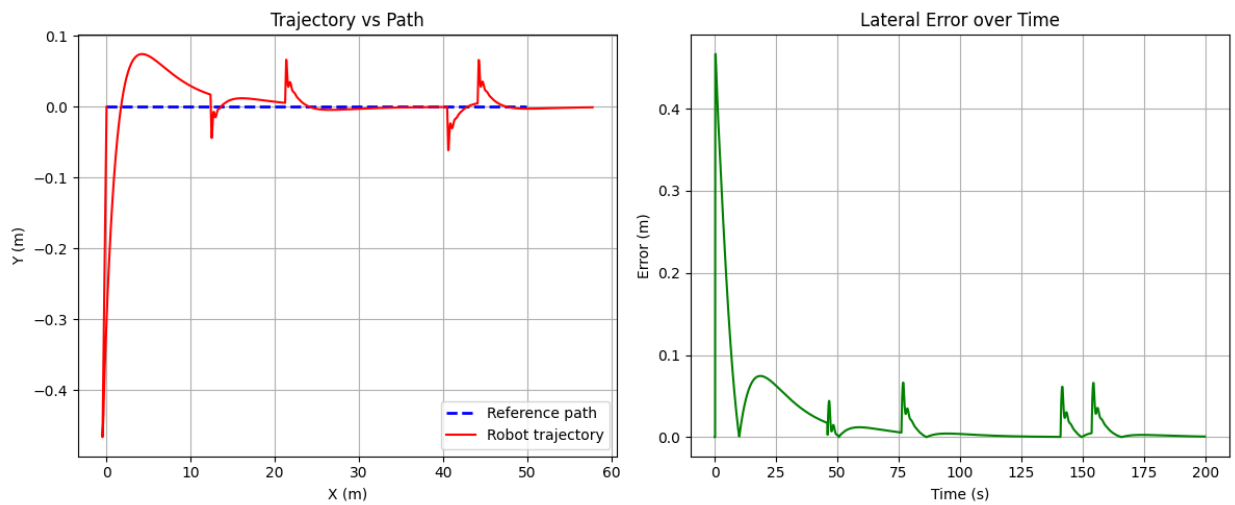


Figure 10 E3 Dist Coeff = 0.5

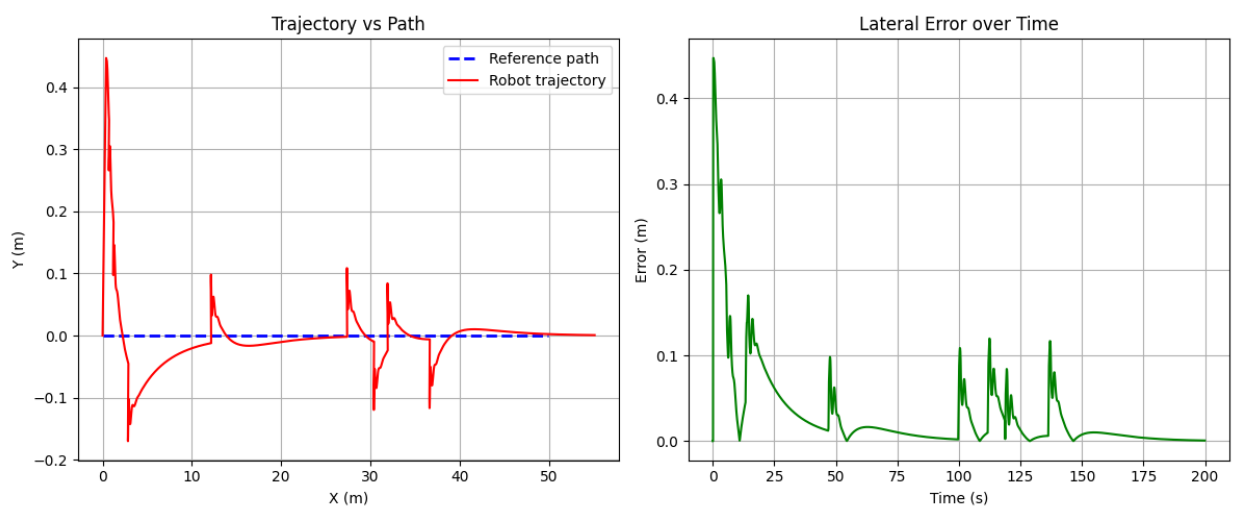


Figure 11 Dist Coeff = 2

Experiment 4 – PD vs PID

A comparison between PD control (Ki=0) and full PID control (Ki=0.1) was conducted on the sinusoidal curved path under noise (σ=0.2). All other gains were kept identical (Kp=2.0, Kd=0.15, Kp_head=2.5).

Results

The PID controller outperformed PD in both overshoot and steady-state error. The integral term reduced steady-state error by 48% (from 0.1272m to 0.0657m) by accumulating and correcting persistent tracking error over time. Under curved path conditions with noise, the accumulated lateral error is more significant, making the integral term especially valuable. Without Ki, the robot exhibits a persistent offset from the curved path that PD alone cannot correct. This confirms the importance of the integral component for accurate curved path tracking under noisy conditions.

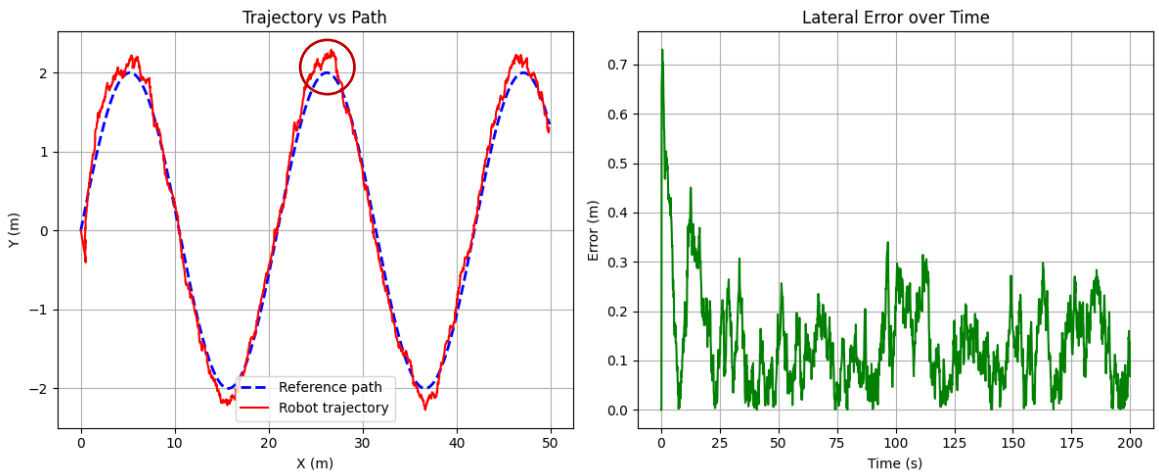


Figure 12 E4 PD

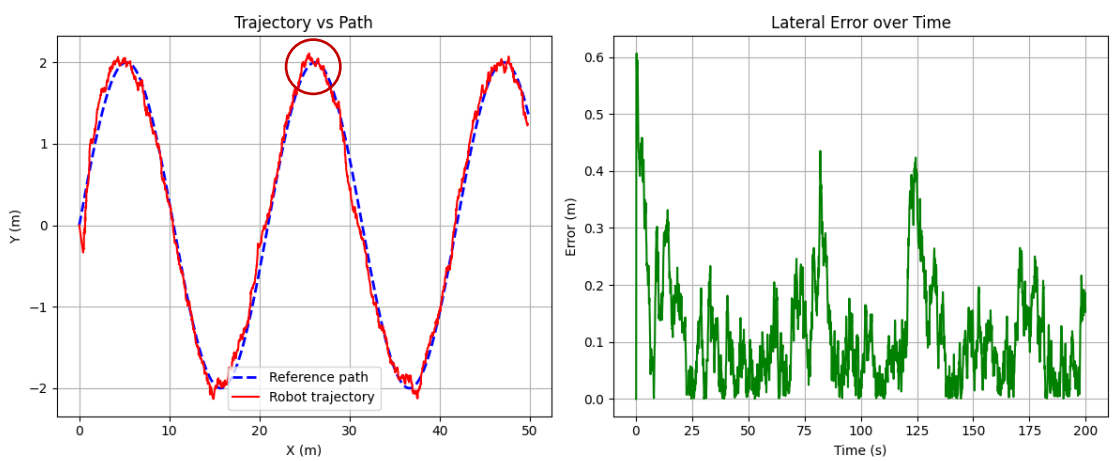


Figure 13 E4 PID

Conclusion

This project successfully implemented a line-following robot digital twin using Siemens Innexis IVSI with three Python clients communicating over CAN protocol. The PID controller with gains Kp=2.0, Ki=0.1, Kd=0.15, Kp_head=2.5 achieved the best performance across all experiments.

Key findings:

- E1: Higher Kp yields faster settling. Integral term eliminates steady-state error on straight paths.
- E2: Curved paths introduce higher tracking error due to continuous curvature changes.
- E3: Controller maintains 100% success rate under all noise/disturbance levels tested.
- E4: PID reduces steady-state error by 48% compared to PD on curved noisy paths.

The IVSI platform proved effective for real-time multi-client simulation, with clean signal exchange over CAN IDs enabling modular development and testing of each system component independently.

Githup Link

https://github.com/Ammar-Wahidi/IVSI-LineFollower-DigitalTwin